

Issue Tracker

Documentation technique complète

Backend Spring Boot · Frontend React · Sécurité JWT · Serveur MCP

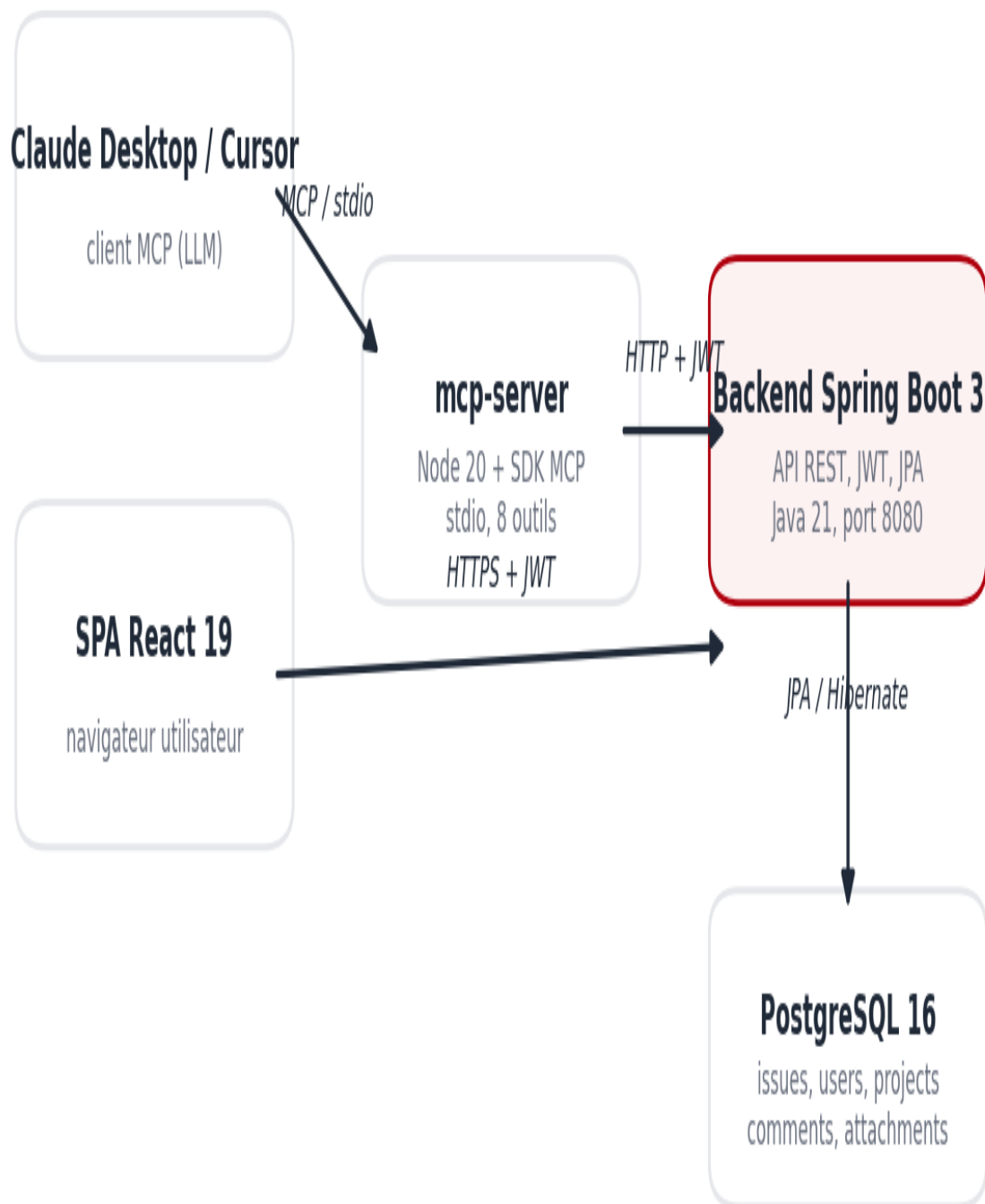
1. Vue d'ensemble du projet

Ce document décrit l'implémentation complète du projet de stage **Issue Tracker** : une application web de gestion de tickets (dans l'esprit de Jira ou GitHub Issues), étendue d'un **serveur MCP (Model Context Protocol)** qui expose ses fonctionnalités à des assistants IA (Claude Desktop, Cursor) sous forme d'outils appelables en langage naturel.

Le projet est structuré en trois modules distincts dans un même dépôt Git :

- **backend/** — API REST Spring Boot 3 (Java 21) + PostgreSQL. C'est le cœur du système : toute la logique métier et toute la sécurité y vivent.
- **frontend/** — SPA React 19 + TypeScript + Vite, design system Tailwind CSS / shadcn/ui. Consomme l'API REST, ne contient aucune règle métier.
- **mcp-server/** — serveur MCP en TypeScript (SDK officiel), couche de traduction fine entre le protocole MCP et l'API REST existante.

Le principe architectural directeur est la **centralisation des règles** : validation, autorisation et transitions de statut n'existent qu'à un seul endroit — le backend. Le frontend et le serveur MCP sont deux clients différents du même contrat REST, comme l'illustre la figure 1.



Un seul contrat REST — deux surfaces clientes (humaine et agent IA).

Figure 1 — Architecture globale : deux clients (SPA React, serveur MCP), un seul contrat REST.

1.1 Périmètre fonctionnel

- Gestion de projets (catégories SOFTWARE / SUPPORT / INTERNAL, leader par projet).
- Cycle de vie complet des issues : création (OPEN), passage IN_PROGRESS, clôture (DONE, terminale), priorités LOW / MEDIUM / HIGH / CRITICAL, assignation multiple.
- Commentaires threadés avec suppression douce (soft delete), pièces jointes (10 Mo max, vérification du type réel par les octets).
- Trois rôles (ADMIN, MANAGER, USER) avec autorisation fine par action.
- Profils utilisateurs (bio, avatar), annuaire d'équipe, administration des rôles.
- Pilotage de tout cela en langage naturel via un assistant IA, grâce au serveur MCP.

2. Stack technique et justification des choix

Couche	Technologie	Rôle	Justification
Backend	Spring Boot 3 / Java 21	API REST, logique métier, sécurité	Écosystème imposé par le plan de stage ; maturité, injection de dépendances, Spring Security.
Persistance	Spring Data JPA (Hibernate) + PostgreSQL	Mapping objet-relationnel, transactions	Base relationnelle adaptée aux relations riches (ManyToMany assignees, threads de commentaires).
Sécurité	Spring Security + JWT (jjwt) + BCrypt	Authentification stateless, autorisation	Stateless = pas de session serveur ; BCrypt = hachage de mots de passe salé, standard industrie.
Frontend	React 19 + TypeScript strict + Vite 8	SPA, UI réactive	Composants typés, build rapide, écosystème dominant.
UI / styles	Tailwind CSS 3 + shadcn/ui (Radix) + lucide-react	Design system	Tokens de design centralisés, primitives accessibles, cohérence visuelle de la maquette de référence.
HTTP front	axios	Client API avec intercepteurs	Intercepteurs requête/réponse = point unique pour le JWT et le rafraîchissement transparent.
MCP server	TypeScript + @modelcontextprotocol/sdk + zod	Serveur MCP	SDK officiel du protocole ; zod = schémas d'entrée validés et convertis en JSON Schema.
Qualité	TypeScript strict, oxlint, JUnit (tests backend)	Robustesse	Typage strict côté front et MCP ; tests controllers/sécurité côté backend.

Tableau 1

3. Backend Spring Boot

3.1 Architecture en couches

Le backend suit une architecture en couches classique et stricte. Chaque requête traverse : **controller** (points d'entrée REST, autorisation déclarative) → **service** (logique métier, transactions) → **repository** (accès JPA) → **entity** (modèle persistant). Les échanges avec l'extérieur passent par des **DTO** dédiés — les entités JPA ne fuient jamais vers l'API. Les erreurs métier sont des exceptions dédiées (*ResourceNotFoundException*, *IssueClosedException*, *CommentDeletedException*, *InvalidRefreshTokenException*) centralisées par un **GlobalExceptionHandler** qui produit une enveloppe uniforme :

```
{ "success": false, "message": "Issue is already closed and cannot change status", "data": null }
```

Cette enveloppe (*GenericType*) est le contrat que tous les clients consomment : le frontend en extrait *message* pour l'afficher, et le serveur MCP le relaie tel quel à l'assistant IA.

3.2 Modèle de données

Six entités principales, reliées comme suit :

- **User** — identifié par un *uuid* public (l'id numérique reste interne), rôle ADMIN / MANAGER / USER, bio, avatar stocké sur disque.
- **Project** — catégorie (SOFTWARE / SUPPORT / INTERNAL), *leader* obligatoire (ManyToOne vers User).
- **Issue** — appartient à un Project ; *creator* (ManyToOne), *assignees* (ManyToMany via la table de jointure *issue_assignees*), *closedBy* + *closedAt* renseignés automatiquement à la clôture.
- **Comment** — appartient à une Issue, auteur, auto-référence *parentComment* pour les réponses threadées, drapeau *deleted* (suppression douce : le fil reste lisible).
- **Attachment** — métadonnées d'un fichier joint à une Issue ; le contenu binaire vit sur disque via *FileStorageService*, jamais en base.
- **RefreshToken** — jetons de rafraîchissement opaques stockés en base (voir section 5).

3.3 Règles métier implémentées dans les services

Les services — et eux seuls — portent les règles. Les plus importantes :

- **Création d'issue** (*IssueService.createIssue*) : le statut est forcé à OPEN et le créateur est lu depuis le SecurityContext (l'utilisateur authentifié) — jamais depuis le corps de la requête. Un client ne peut donc ni falsifier son identité ni créer une issue directement DONE.
- **Clôture terminale** (*updateStatus*, *updateIssue*) : une issue DONE est verrouillée — toute modification ou transition ultérieure lève *IssueClosedException* (HTTP 409). La clôture horodate *closedAt* et mémorise *closedBy*.
- **Mise à jour d'issue** : le statut ne passe JAMAIS par PUT /issues/{id} ; seul PATCH /issues/{id}/status peut le changer (séparation des responsabilités).
- **Projet** : la catégorie ne change que via PATCH /projects/{id}/category, réservé à l'ADMIN.

- **Commentaires** : la suppression est douce — le contenu est remplacé par « [comment deleted] » mais le fil de discussion reste cohérent.
- **Fichiers** : 10 Mo max par pièce jointe, type réel vérifié par les octets (Apache Tika) contre une liste blanche ; avatar 3 Mo, images uniquement.

3.4 Surface de l'API REST

Domaine	Endpoints principaux	Accès
Auth	POST /api/auth/register · login · refresh · logout	public
Issues	GET/POST /api/issues · GET/PUT/DELETE /api/issues/{id} · PATCH /api/issues/{id}/status	authentifié ; écriture selon rôle + créateur/assigné/leader
Commentaires	POST /api/issues/{id}/comments · PUT/DELETE /api/comments/{id}	authentifié ; édition = auteur ou staff
Pièces jointes	POST /api/issues/{id}/attachments · GET/DELETE /api/attachments/{id}/(content)	authentifié ; suppression = uploader ou staff
Projets	GET/POST/PUT/DELETE /api/projects · PATCH /api/projects/{id}/category	lecture : tous ; écriture : staff ; catégorie/suppression : ADMIN
Utilisateurs	GET /api/users/me · PATCH /users/me · POST /users/me/avatar · GET /users/search · GET /users/{uuid}/profile · GET /api/users · PATCH /users/{uuid}/role	annuaire : tous ; liste complète et rôles : ADMIN

Tableau 2

La liste des issues est paginée et filtrable (projet, statut, priorité, assigné) avec tri sur quatre champs autorisés (*status*, *priority*, *createdAt*, *updatedAt*) — la liste blanche `SORTABLE_FIELDS` de `IssueService` empêche l'injection de champs de tri arbitraires.

4. Frontend React

4.1 Stack et design system

Le frontend est une SPA React 19 / TypeScript strict / Vite 8. La couche visuelle suit une maquette de référence imposée : **Tailwind CSS 3** pour l'utilitaire, **shadcn/ui** (primitives Radix accessibles) pour les composants, **lucide-react** pour les icônes, **sonner** pour les notifications toast. Les tokens de design sont centralisés en variables CSS HSL dans `src/index.css` : primaire #E60012, rouge foncé #B0000E, fond #F7F8FA, bordures #E5E7EB — toute l'application en dérive, ce qui garantit la cohérence visuelle.

4.2 Structure du code

```
src/
■■■ main.tsx          # RouterProvider + <Toaster/>
■■■ App.tsx           # garde d'authentification, overlays globaux (form + détail d'issue)
■■■ routes/router.tsx # table de routage
■■■ types/            # types du domaine (timestamps en ms)
```

```

■■■ store/AppStore.tsx # store unique (contexte React) – voir 4.3
■■■ lib/
■   ■■■ api.ts         # tous les appels backend + mapping DTO → types UI
■   ■■■ permissions.ts # gates d'UI par rôle (miroir de l'autorisation backend)
■   ■■■ helpers.ts     # timeAgo, formatDate, libellés, ordres de tri
■   ■■■ issueQuery.ts  # filtre/tri côté client de la liste d'issues
■■■ components/        # AppShell, KanbanBoard, IssueTable, FilterBar, IssueDetail,
■   ■■■ ui/            # IssueForm, ProjectForm, AssigneePicker, bits + primitives shadcn
■■■ pages/             # Auth, Dashboard, BoardPage, ProjectPage, TeamPage, Profiles, AdminUsers
■■■ utils/
    ■■■ apiClient.ts   # axios : JWT, refresh single-flight, 401 → reconnexion
    ■■■ apiTypes.ts    # enveloppes GenericResponse / PagedResponse

```

4.3 Le store applicatif (AppStore)

`src/store/AppStore.tsx` est la source de vérité unique (contexte React). Son interface est calquée sur celle de la maquette de référence, mais chaque méthode est adossée à l'API réelle :

- **Amorçage de session** : au montage, le token persisté est validé via `GET /users/me` ; puis hydratation complète (projets, utilisateurs, issues — boucle de pagination de 200).
- **Chargement paresseux du détail** : commentaires et pièces jointes ne sont chargés (`GET /issues/{id}`) qu'à l'ouverture d'une issue.
- **Mises à jour optimistes avec rollback** : le changement de statut (`PATCH /issues/{id}/status`) et le changement de rôle (`PATCH /users/{uuid}/role`) sont appliqués localement d'abord ; en cas d'échec serveur, l'état antérieur est restauré et un toast d'erreur s'affiche. L'UI reste instantanée sans jamais mentir durablement.
- **Échec d'authentification** : tout 401 survivant à la tentative de rafraîchissement déclenche la même déconnexion propre que le bouton « Sign Out » (jetons effacés, redirection `/login`).

4.4 Le client HTTP (apiClient.ts)

Toute la machinerie JWT côté client tient en un module, `src/utils/apiClient.ts` :

- **Intercepteur de requête** : injecte `Authorization: Bearer <token>` depuis `localStorage` sur chaque appel.
- **Rafraîchissement « single-flight »** : sur 401, un seul `POST /auth/refresh` est émis ; toutes les requêtes concurrentes attendent la même promesse — crucial car le refresh token est **à usage unique** côté serveur (deux rafraîchissements parallèles avec le même jeton feraient rejeter le second).
- **Retry transparent** : la requête d'origine est rejouée avec le nouveau token ; l'utilisateur ne voit rien.
- **Discrimination des 401** : les 401 de `/auth/login` (mauvais identifiants) et `/auth/refresh` (jeton mort) ne déclenchent PAS la déconnexion globale — ce sont des résultats normaux de formulaire, gérés localement.
- **getErrorMessage** : extrait le *message* de l'enveloppe d'erreur du backend pour l'afficher tel quel dans l'UI.

4.5 Routage et rétro-compatibilité

Chemin	Page	Note
/login · /register	Auth	redirige vers /dashboard si déjà connecté
/dashboard	Dashboard	/ redirige ici (ancien signet)
/board	Issue Board	vues kanban + table
/projects/:projectId	ProjectPage	chemin historique préservé
/team	TeamPage	/users/search redirige ici
/profile	MyProfilePage	/profile/edit redirige ici
/users/:uuid	UserProfilePage	/profile/:uuid redirige ici
/admin/users	AdminUsersPage	réservé ADMIN (UI + backend)

Tableau 3

Les **gates de permission** (*lib/permissions.ts*) masquent ou verrouillent les actions non autorisées (jamais de bouton cassé) ; elles sont un confort d’UI — l’autorité reste le backend.

5. Sécurité : JWT et autorisation

5.1 Le flux d’authentification complet

L’authentification est **stateless** : aucune session serveur. Le flux, de bout en bout :

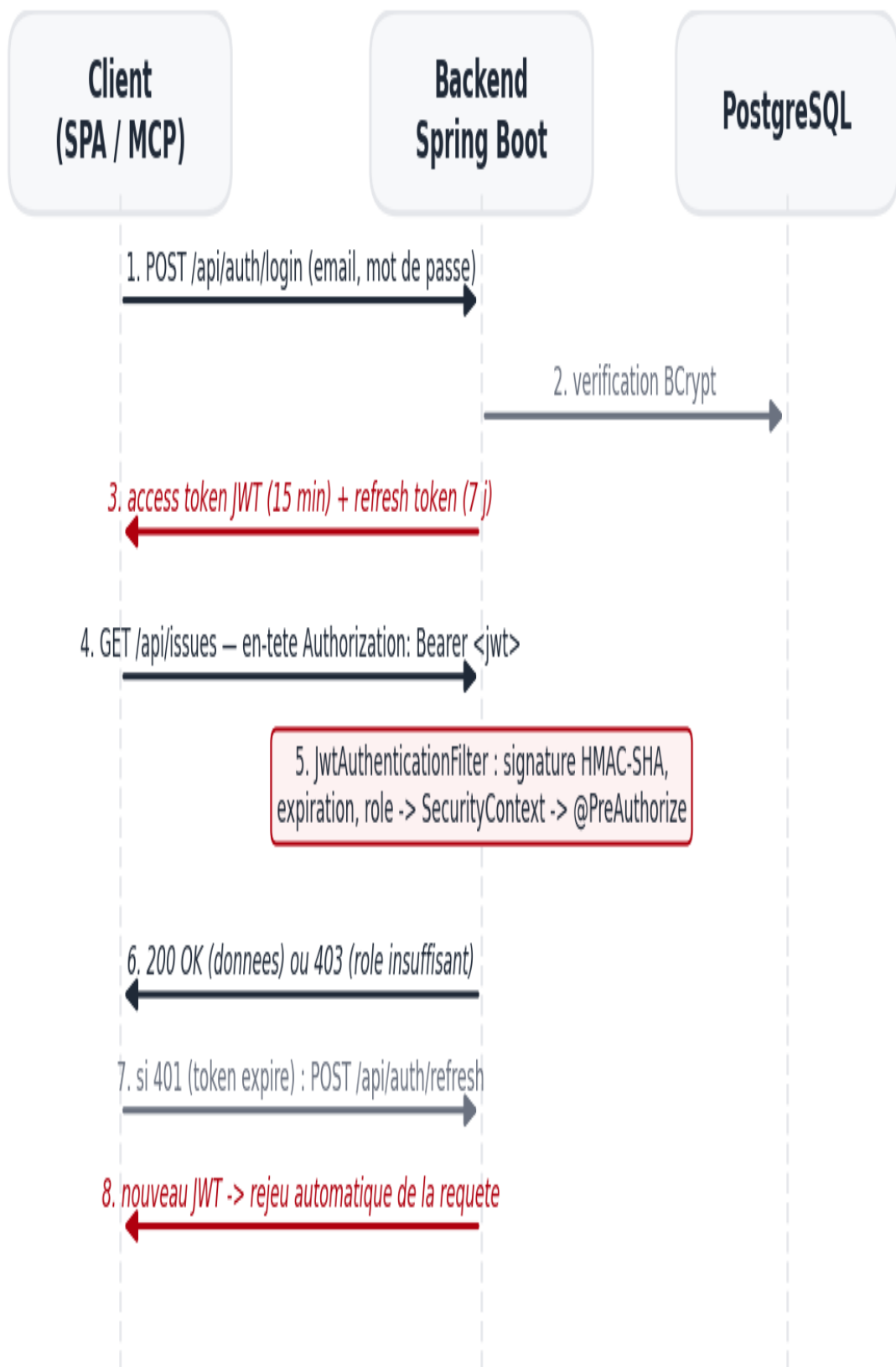


Figure 2 — Flux JWT : connexion, appel authentifié, rafraîchissement transparent.

- **Connexion** (POST /api/auth/login) : *AuthenticationManager* vérifie le mot de passe via **BCrypt** (hachage salé — la base ne stocke jamais de mot de passe en clair). En succès, *JwtService* signe un **access token** (HMAC-SHA256, clé secrète fournie par la variable d'environnement `JWT_SECRET` — sans valeur par défaut : l'application refuse de démarrer sans elle) et *RefreshTokenService* émet un **refresh token opaque** aléatoire, persisté en base.
- **Appel authentifié** : *JwtAuthenticationFilter* intercepte chaque requête, valide la signature et l'expiration du JWT, charge l'utilisateur et peuple le **SecurityContext**. C'est de ce contexte — jamais de la requête — que les services tirent l'identité de l'appelant.
- **Rafraîchissement** (POST /api/auth/refresh) : le refresh token est à **usage unique** (*verifyAndConsume*) : chaque rotation invalide le précédent — un jeton volé et rejoué est détecté et rejeté. La déconnexion (POST /auth/logout) révoque le jeton côté serveur.
- **Autorisation** : *SecurityConfig* rend /api/auth/** public et exige l'authentification sur tout le reste de /api/** ; au-delà, **@EnableMethodSecurity** active des gardes fines **@PreAuthorize** sur chaque endpoint sensible.

5.2 La matrice d'autorisation

Trois beans dédiés (*IssueSecurity*, *CommentSecurity*, *AttachmentSecurity*) expriment les règles contextuelles (créateur, assigné, leader de projet) ; les règles de rôle pur sont déclarées inline :

Action	Règle exacte (backend)
Créer une issue	tout utilisateur authentifié
Commenter / changer le statut	ADMIN, MANAGER, ou créateur/assigné de l'issue
Éditer les champs / réassigner / attacher	les mêmes + le leader du projet
Supprimer une issue	ADMIN ou MANAGER
Créer / éditer un projet	ADMIN ou MANAGER
Supprimer un projet / changer sa catégorie	ADMIN uniquement
Lister tous les utilisateurs / changer un rôle	ADMIN uniquement
Supprimer un commentaire / une pièce jointe	auteur (ou uploader) ou staff

Tableau 4

Point clé pour la suite : ces règles s'appliquent à **tout appelant HTTP** — navigateur du frontend, script curl, ou serveur MCP. Aucun client ne peut les contourner sans contourner l'API elle-même.

6. Le serveur MCP (partie centrale)

6.1 Qu'est-ce que MCP, concrètement ?

Le **Model Context Protocol** est un protocole ouvert (JSON-RPC 2.0) qui standardise la conversation entre un **client IA** (Claude Desktop, Cursor...) et un **serveur** que l'on écrit. Le serveur expose trois primitives : les **tools** (actions que le modèle décide d'appeler), les **resources** (données en lecture) et les **prompts** (modèles de conversation). Ce projet n'utilise que des tools — chacun correspond à une opération métier de l'Issue Tracker.

Le cycle de vie, tel qu'implémenté et vérifié : au démarrage, le client lance le serveur et appelle *initialize* puis *tools/list* — le serveur répond avec les noms, descriptions et schémas JSON de ses outils, que le modèle garde en contexte. Quand l'utilisateur formule une demande (« crée une issue dans le projet Mobile App »), le modèle choisit un outil, remplit les arguments, et le client émet *tools/call* ; le serveur exécute et renvoie le résultat — ou l'erreur.

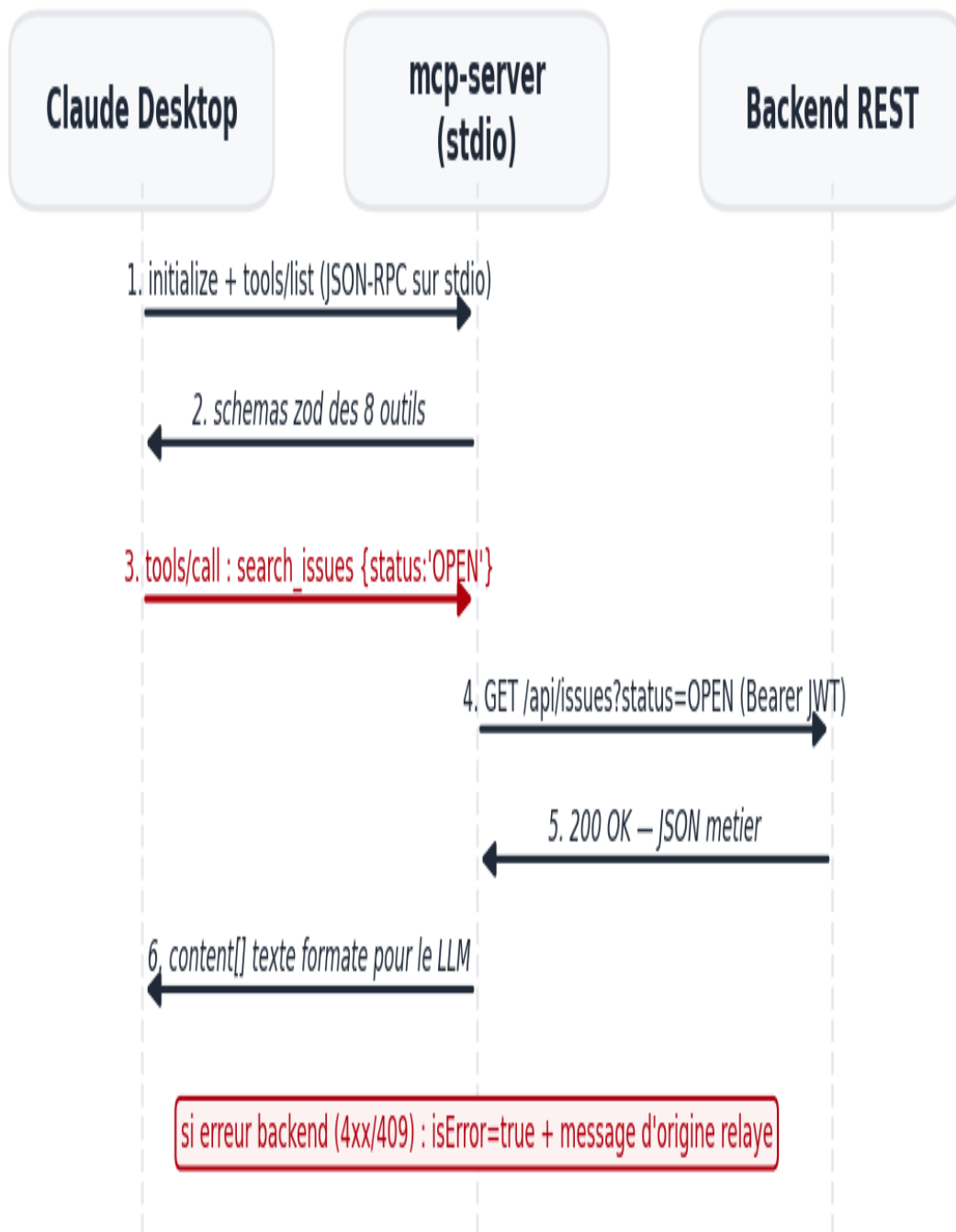


Figure 3 — Cycle de vie d'un appel MCP : découverte puis exécution.

Deux transports existent : **stdio** (le client lance le serveur en sous-processus et parle sur stdin/stdout — aucun port réseau, aucune couche d'authentification à inventer) et **Streamable HTTP** (serveur réseau, pour du multi-utilisateurs distant). Le projet utilise stdio : c'est le mode standard des serveurs MCP personnels, et le code des outils est indifférent au transport.

6.2 Décision d'architecture : pourquoi une couche fine au-dessus de l'API

Option	Principe	Verdict
A — MCP dans l'app Spring Boot	le serveur MCP vit dans la même JVM et appelle les services en direct	rejetée : couplage fort, et le SecurityContext (identité de l'appelant) n'existe pas hors d'une requête HTTP — il aurait fallu le bricoler, au risque de court-circuiter @PreAuthorize
B — MCP séparé appelant l'API REST	processus indépendant, client HTTP authentifié par JWT	RETENUE : zéro duplication de logique, autorisation backend intacte, identité naturelle via JWT, cycle de vie indépendant
C — module Java partageant les services	deux artefacts, code métier partagé	rejetée : réintroduit le couplage et le problème du SecurityContext

Tableau 5

L'option B fait de l'API REST le **contrat unique**. Le serveur MCP ne contient aucune règle : il déclare des outils avec des schémas stricts, appelle l'API, et traduit fidèlement réponses et clients erreurs. Les @PreAuthorize, les validations de DTO et les règles de transition continuent de s'appliquer — gratuitement.

6.3 Implémentation : structure et code

Le module *mcp-server/* tient en deux fichiers source :

```
mcp-server/
■■■ src/
■   ■■■ api.ts      # client HTTP : config env, cycle de vie JWT, erreurs
■   ■■■ index.ts    # serveur MCP : déclaration des 8 outils
■■■ test-e2e.mjs    # test de bout en bout (pilote le serveur en stdio)
■■■ README.md       # installation + configuration Claude Desktop / Cursor
■■■ *.example.json  # configurations d'exemple pour les deux clients
```

6.3.1 api.ts — le client API et le cycle de vie du JWT

Le serveur s'authentifie comme un **compte de service** dédié (utilisateur réel en base, ici *claude.bot*, rôle *MANAGER*). Trois variables d'environnement le configurent : *ISSUE_TRACKER_API_URL*, *ISSUE_TRACKER_USERNAME*, *ISSUE_TRACKER_PASSWORD*. Le module gère le cycle de vie du JWT de façon autonome :

- **Login paresseux** : la première requête déclenche POST /auth/login ; les jetons sont gardés en mémoire de processus.
- **401 → refresh → retry** : sur un 401, le serveur tente POST /auth/refresh (rotation à usage unique gérée côté serveur), retombe sur un login complet si le refresh a expiré, puis rejoue la requête une seule fois.
- **ApiError** : toute erreur HTTP devient une exception typée portant le statut et le *message* du backend — la matière première du relayage d'erreurs (6.4).

```
if (res.status === 401 && !isRetry) {
  await refresh() // POST /auth/refresh, sinon login()
  return apiFetch<T>(method, path, body, true) // un seul retry
}
if (!res.ok) throw new ApiError(res.status, json.message ?? `HTTP ${res.status}`)
```

6.3.2 index.ts — anatomie d'un outil

Chaque outil est déclaré via `server.registerTool` avec quatre morceaux : un **nom**, une **description** en langage naturel (le texte que le modèle lit pour décider quand et comment appeler l'outil — c'est l'interface avec l'IA), un **inputSchema** en zod (converti en JSON Schema et validé avant tout appel du handler), et le **handler** qui appelle l'API. Exemple réel :

```
server.registerTool('update_issue_status', {
  title: 'Update issue status',
  description:
    'Change the status of an issue. Valid statuses: OPEN, IN_PROGRESS, DONE. ' +
    'DONE is terminal: a DONE issue is locked and cannot be edited or reopened. ' +
    'The backend enforces who is allowed to change status and rejects invalid transitions.',
  inputSchema: {
    issueId: z.number().int().describe('Numeric id of the issue'),
    status: z.enum(['OPEN', 'IN_PROGRESS', 'DONE']).describe('New status'),
  },
}, async ({ issueId, status }) =>
  run(() => apiFetch('PATCH', `/issues/${issueId}/status`, { status })))
```

Notez que la description **enseigne la règle métier au modèle** (« DONE is terminal ») : l'IA est prévenue avant d'agir, et le backend reste le juge si elle essaie quand même.

6.4 Les huit outils exposés

Outil	Rôle	Endpoint backend
list_projects	liste les projets (trouver le projectId)	GET /api/projects
search_issues	filtre les issues (projet, statut, priorité, assigné, pagination)	GET /api/issues
get_issue	détail complet : commentaires threadés + pièces jointes	GET /api/issues/{id}
search_users	résout une personne en uuid (trouver les assignedUuids)	GET /api/users/search
create_issue	crée une issue (statut OPEN imposé par le backend)	POST /api/issues
update_issue_status	OPEN / IN_PROGRESS / DONE (DONE terminale)	PATCH /api/issues/{id}/status
assign_issue	remplace les assignés (fusion avec l'état courant, PUT)	PUT /api/issues/{id}
add_comment	commente, réponses threadées possibles	POST /api/issues/{id}/comments

Tableau 6

Gestion des erreurs. Toute erreur backend (403 interdit, 404 introuvable, 409 issue verrouillée, 400 validation) est renvoyée au client IA comme **erreur d'outil** (drapeau `isError` du protocole) avec le message exact du backend. L'assistant peut alors expliquer le refus à l'utilisateur au lieu d'échouer silencieusement — c'est ce qui rend les appels « fiables » au sens des critères d'évaluation.

Anti-hallucination. Les outils de lecture existent *avant* les outils d'écriture par design : les descriptions de `create_issue` et `assign_issue` ordonnent littéralement au modèle d'appeler `list_projects / search_users` d'abord (« never guess a project id »). Les identifiants inventés qui passeraient quand même sont rejetés par le backend (404) et relayés.

6.5 Sécurité du dispositif MCP

- **Le compte de service est le plafond des pouvoirs de l'IA.** `claude.bot` est MANAGER : l'assistant peut gérer projets et issues, mais ne pourra jamais changer un rôle ni supprimer un projet (règles ADMIN) — le backend répondrait 403, relayé comme erreur.
- **Aucune règle n'est dupliquée.** La sécurité effective reste à 100 % dans le backend ; le MCP n'ajoute que des garde-fous d'entrée (schémas zod) et de bonnes descriptions.
- **Traçabilité.** Chaque action de l'IA est attribuée à une identité réelle (créateur, `closedBy`, auteur des commentaires = `claude.bot`), visible dans l'UI et les données.
- **studio = pas de surface réseau.** Le serveur n'écoute aucun port ; seul le client qui l'a lancé peut lui parler.

6.6 Configuration des clients et vérification

Claude Desktop lit `claude_desktop_config.json`, Cursor lit `~/.cursor/mcp.json` ; les deux déclarent comment lancer le serveur et injectent les variables d'environnement :

```
{
  "mcpServers": {
    "issue-tracker": {
      "command": "node",
      "args": ["../mcp-server/dist/index.js"],
      "env": {
        "ISSUE_TRACKER_API_URL": "http://localhost:8080/api",
        "ISSUE_TRACKER_USERNAME": "claude.bot",
        "ISSUE_TRACKER_PASSWORD": "..."
      }
    }
  }
}
```

La vérification a été faite de bout en bout par un test `studio` (`test-e2e.mjs`) qui pilote le serveur exactement comme un client MCP : handshake, découverte, puis appel de chaque outil contre le backend réel. Résultat : **12/12 contrôles réussis**, y compris les deux contrôles de règles métier — la réouverture d'une issue DONE est bien rejetée (« Backend error 409: Issue is already closed ») et un `projectId` inconnu renvoie 404, tous deux relayés comme erreurs d'outil.

7. Lancer l'ensemble du système

```
# 1. Base de données (PostgreSQL sur localhost:5432)
CREATE DATABASE issuetracker;

# 2. Backend (terminal 1) – JWT_SECRET est obligatoire
export JWT_SECRET=$(openssl rand -base64 32)
cd backend && ./mvnw spring-boot:run # http://localhost:8080

# 3. Frontend (terminal 2)
```

```
cd frontend && npm install && npm run dev      # http://localhost:5173 (proxy /api)

# 4. Serveur MCP (lancé automatiquement par le client IA)
cd mcp-server && npm install && npm run build
#   puis déclarer dist/index.js + les variables d'env dans Claude Desktop / Cursor
```

8. Limites connues et évolutions

- **Transport HTTP** : pour un usage multi-utilisateurs ou distant, le même code d'outils peut être servi via Streamable HTTP (avec une couche d'authentification MCP dédiée).
- **Backend déployé** : pointer `ISSUE_TRACKER_API_URL` vers l'URL de production suffit — le backend possède déjà un profil prod.
- **Resources et prompts MCP** : non utilisés aujourd'hui ; des resources en lecture seule (ex. « l'issue ouverte ») seraient un ajout naturel.
- **Outillage d'écriture supplémentaire** : édition d'issue complète, gestion de projets — le même patron d'outil s'applique.